Strangler Fig Pattern

Category: Migration / modernization **One-liner:** Replace an old system gradually by putting a router in front of it and moving one capability at a time to the new system, instead of rewriting everything and switching over in one go.

Part 0 — Concepts you need first

If these are already familiar, skip to Part 1.

What is a monolith?

A **monolith** is an application built and deployed as one single unit. All the code — user accounts, payments, reporting, email — lives in one **codebase** (the full set of source files for a program), compiles into one **artifact** (the finished, runnable package the build produces, such as a `.jar` file), and runs as one **process** (a single running program instance on a machine, with its own memory). Changing one line in the reporting module means rebuilding and **redeploying** (putting a new version into the live environment) *everything*, including payments.

Monoliths are not evil. For a small application a monolith is usually the *right* choice — it's simpler to build, test, and deploy. The problems start when it grows large and many teams need to change it simultaneously.

What is a legacy system?

"Legacy" doesn't just mean old. It means **old software that is still running because it does something essential**. That's the crucial part: it survives precisely because the business depends on it. Nobody keeps unimportant old software running; they delete it.

This creates the trap that defines this whole chapter: the system is **too risky to keep** (it's fragile and expensive) and **too risky to replace** (it's load-bearing).

What is a "big-bang rewrite"?

The instinctive approach to a bad old system: build a complete replacement, then switch everything over on one date. It's called big-bang because everything changes at once.

It fails often, and for a predictable reason. While you spend eighteen months building the replacement, the old system keeps changing — bugs get fixed, features get added, regulations change. You're rebuilding a moving target. On switchover day you discover the hundred behaviours nobody documented, and you have no way back.

What is a proxy or router (in this context)?

A piece of software sitting **between** a client and one or more servers. The client talks only to the proxy; the proxy decides where the request actually goes. The client cannot tell the difference.

This "client can't tell the difference" property is the entire foundation of the Strangler Fig Pattern. If you can put something in the middle that clients can't distinguish from the real system, you can change what's behind it whenever you like.

What is a feature flag?

A configuration switch that turns behaviour on or off **without deploying new code**. Instead of:

```
return newImplementation(x);
```

you write:

```
if (flags.isEnabled("use-new-implementation")) {
    return newImplementation(x);
}
return oldImplementation(x);
```

Now you can change behaviour by editing configuration, and change it *back* in seconds if something breaks. Flags are what make the migration reversible.

Part 1 — Where the name comes from

The **strangler fig** is a real tree. Its seed germinates in the branches of an existing tree. It grows roots downward and branches upward, gradually enveloping its host. Eventually the host tree dies and rots away, leaving the fig standing in its exact shape — hollow in the middle where the original used to be.

Two things in that image matter for software:

1. **The fig grows around the host while the host is still alive.** It doesn't chop the tree down and plant a seedling in the gap. Both exist simultaneously for a long time.
2. **The transition is gradual and continuous.** There is no single moment when the tree becomes a fig.

Applied to software: your new system grows *around* the legacy system, which continues serving real **production** traffic (requests from real users hitting the live system, as opposed to test environments) the entire time it's being displaced.

The course extends the metaphor: picture an ancient, enormous tree in a dense forest. That's your legacy system — rooted deeply in the organisation's architecture, its many branches representing all the functions it supports. Replacing it feels as daunting as replacing that tree.

Part 2 — The problem in detail

Why legacy systems are painful to live with

Technical debt. Over years, patches, fixes, and new features get added haphazardly — often under deadline pressure, often by people who have since left. The result is a complex, hard-to-maintain codebase where no one is confident what a change will affect.

> *Beginner note:* "technical debt" is a metaphor — taking a shortcut now (like debt) lets you move fast today but charges interest later in the form of slower future changes. Some debt is a reasonable trade; the problem is debt that's never repaid.

Obsolete technology. The **stack** (the set of languages, frameworks, databases and servers the system is built on) is no longer current. Concretely, this means you can't hire people who know it (or must pay a large premium), the vendor no longer issues support or **patches** (small updates that fix bugs or security holes), and the **libraries** (reusable code packages written by others that your program calls into) you depend on have unfixed bugs forever.

Integration issues. Modern tools and services don't integrate easily with older systems. The practical consequence is **silos of information** — data trapped in a system that nothing else can conveniently read, so it gets manually exported, copied, and duplicated.

Scalability concerns. (**Scalability** is a system's ability to handle more work by adding resources; **load** is the amount of work — requests, users, data — hitting it.) The system was designed for the load and deployment model of its era. It may assume a single server, or a fixed number of them. It's typically **not cloud-ready** (the **cloud** being rented, on-demand servers from providers such as AWS, Azure or Google Cloud), meaning it can't simply be run as more copies when traffic grows.

Costly maintenance. As the system ages, maintenance gets more expensive in both time and resources. Every change takes longer because the code is tangled and the people who understand it are fewer.

Security vulnerabilities. It was designed before current security practices existed. It may store passwords insecurely, lack protection against now-standard attacks, or run on an **OS** (operating system — e.g. Windows or Linux, the base software that runs the machine) that no longer receives patches.

Why legacy systems are painful to replace

Critical operations. These systems are deeply embedded in the organisation's critical operations. Any disruption has significant impact — this is not "the site is slow," it's "we cannot take orders today."

Migration risks. Moving data from the old system to the new risks **data loss or corruption**. Old **schemas** (the defined structure of a database: its tables, columns and their types) usually contain years of accumulated irregularity: records that violate current rules, fields reused for different purposes, **encoding** inconsistencies (text stored using different rules for turning characters into bytes, so accents or symbols come out garbled).

Training and adoption. Staff know the old system. A new one requires training and often meets genuine resistance — people who were fast and confident are suddenly slow and uncertain.

High costs. Building a new system *and making it as reliable as the old one* is expensive. That second part is easy to underestimate: the old system's reliability came from a decade of bugs found and fixed in production. Your new system hasn't had those years.

Unknown unknowns. There are undocumented features and behaviours that users have come to depend on.

> This is the one that kills rewrites. You cannot write a specification for behaviour nobody wrote down and nobody remembers. You discover it when you switch over and someone says "the report used to round differently and now our reconciliation breaks." A rewrite finds these all at once, on the worst possible day.

Part 3 — The solution

Don't replace the system. **Strangle it.**

Build the new system alongside the old one. Put a routing layer in front of both. Migrate one capability at a time, flipping the router as each new piece proves itself in production. When no traffic reaches the legacy system anymore, delete it.

Why this beats a rewrite

Less risky. The old system is still there. Each step has a safety net, and each step is individually revertible. If the new checkout breaks, you route checkout back to the old system in seconds.

Flexible prioritisation. You migrate by **urgency or business need**, not by the dependency order a rewrite would force. If the payment module is the most painful, do it first.

No big-bang releases. Continuous delivery (keeping software always releasable and shipping small changes frequently) and gradual replacement instead of one catastrophic **cutover** date (the moment traffic is switched from the old system to the new). Less potential for massive failures.

Smooth transition. Done well, users may not notice a shift happened at all — it's gradual and each piece is tested before it carries real traffic.

Early feedback loops. Real users hit new code early, so you learn about problems while they're still cheap to fix, rather than discovering everything at once at the end.

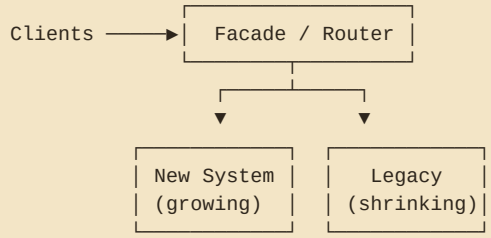
Technology flexibility. The new system can adopt new technology as the industry evolves. You're not locked into whatever was fashionable at project kickoff — a real concern when a migration takes two years.

Part 4 — Architecture

The Facade Interface — the gatekeeper

The central component. Every incoming request hits the facade first, and it makes one decision: **has this functionality been migrated yet?**

- **Yes** → route to the new system
- **No** → route to the legacy system



The course's analogy: **a traffic controller at an intersection**, guiding cars (requests) down the correct road.

The critical property: **clients are unaware there are two systems at all.** They see one address, one **API** (Application Programming Interface — the set of requests a system accepts and the responses it returns), one system. That's what makes the migration invisible.

> *Beginner note — why "facade"?* A facade is the front of a building. In software, the Facade pattern means putting a simple, unified front over something complicated behind it. Here the complicated thing is "there are two entire systems and the answer depends on which feature you want."

Layered, phased migration

The course describes it as **peeling an onion layer by layer**, each layer being a part of the legacy system replaced by the new one. The migration happens step by step, never all at once:

1. **Identify a piece of functionality that can be cleanly isolated** and redirected.
2. **Rebuild that functionality** in the new system.
3. **Test it fully.**
4. **Flip the facade** to route that functionality to the new system, while everything else still goes to legacy.
5. **Repeat.**

That gradual shift of responsibility from old to new is the crux of the pattern.

Step 1 is the hard one, and beginners underestimate it. "Cleanly isolated" means the feature doesn't share **mutable state** (data that can be changed while the program runs) with everything else. In practice most legacy systems have one giant shared database where everything touches everything, so finding a genuinely separable **seam** (a boundary where one part can be cut away from the rest with few connections across it) is the real work. Look for capabilities that read a lot and write little, or that own a table nothing else writes to.

Part 5 — The four implementation steps

1. **Identify entry points** — choose the parts of the legacy system you want to replace, and build new services to take over those functions.
2. **Route traffic** — use a router or proxy to direct traffic to either the old system or the new service, based on what's been migrated so far.
3. **Incrementally build and replace** — develop the new services and slowly displace old functionality.

4. **Retire the old** — once all functionality has been taken over, safely **decommission** (permanently shut down and remove) the old system.

Part 6 — Worked example 1: routing a single function

The scenario: an old user-management system. You want to replace just `getUserDetails`.

```
class UserServiceRouter:
    def get_user_details(self, user_id):
        if self.use_new_service(user_id):
            return NewUserService().get_user_details(user_id)
        else:
            return OldUserService().get_user_details(user_id)

    def use_new_service(self, user_id):
        # Logic deciding which service to use.
        # For simplicity: use the new service for even user IDs.
        return user_id % 2 == 0
```

Reading this line by line

- `UserServiceRouter` is the facade. **Callers use this class, not the two services directly** — that's what makes the switch invisible to them.
- `get_user_details` has the same **signature** (the method's name, parameters and return type) as the underlying services. The router is a **drop-in replacement** (something callers can swap in without changing their own code) for whatever callers used before.
- `use_new_service` is the **decision function** — the entire migration policy lives in this one method.
- `user_id % 2 == 0` means "even user IDs." This sends roughly 50% of users to the new service.

Why the even-ID trick is smarter than it looks

This isn't just a toy. It gives you a **percentage rollout dial**.

> **Key term — hash function:** A **hash function** turns an input (like a user ID) into a number in a scrambled but repeatable way: the same input always produces the same output, and different inputs spread fairly evenly across the possible outputs. Taking `hash(user_id) % 100` (the remainder after dividing by 100) therefore puts each user in a fixed "bucket" from 0 to 99. You're not choosing "old or new" — you're choosing *how much* new:

```
return hash(user_id) % 100 < 5     # 5% of users
return hash(user_id) % 100 < 50    # 50% of users
return True                        # everyone
```

And it's **consistent per user**: user 42 always gets the same implementation. That matters enormously — if users flipped between implementations randomly on each request, any difference in behaviour would look like a bizarre intermittent bug, and you'd never reproduce it.

As confidence grows, widen the condition until the old path is **dead code** (code that still exists but can never run) — then delete it.

Part 7 — Worked example 2: a facade over a banking system

A fuller example. The legacy system has four operations:

```
public class LegacySystem {
    public void createAccount(String name) { /* old logic */ }
    public void deposit(int amount)        { /* old logic */ }
    public void withdraw(int amount)       { /* old logic */ }
    public int  checkBalance()             { /* old logic */ }
}
```

The new system implements the same four operations with updated logic:

```
public class NewSystem {
    public void createAccount(String name) { /* new logic */ }
    public void deposit(int amount)        { /* new logic */ }
    public void withdraw(int amount)       { /* new logic */ }
    public int  checkBalance()             { /* new logic */ }
}
```

Notice the signatures are identical. That's deliberate and necessary — the facade can only substitute one for the other if they're interchangeable.

Now the facade — the class the outside world actually talks to. It holds an instance of both systems and directs each request:

```
public class BankingFacade {
    private LegacySystem legacySystem;
    private NewSystem newSystem;

    public BankingFacade(LegacySystem legacySystem, NewSystem newSystem) {
        this.legacySystem = legacySystem;
        this.newSystem = newSystem;
    }

    public void createAccount(String name) {
        if (newSystemReadyFor("createAccount")) {
            newSystem.createAccount(name);
        } else {
            legacySystem.createAccount(name);
        }
    }

    public void deposit(int amount) {
        if (newSystemReadyFor("deposit")) {
            newSystem.deposit(amount);
        } else {
            legacySystem.deposit(amount);
        }
    }

    public int checkBalance() {
        if (newSystemReadyFor("checkBalance")) {
            return newSystem.checkBalance();
        } else {
            return legacySystem.checkBalance();
        }
    }

    private boolean newSystemReadyFor(String functionality) {
        // Check whether the new system is ready for this functionality
        return false;   // placeholder — starts false for everything
    }
}
```

How the migration actually proceeds

At the start, `newSystemReadyFor` returns `false` for everything, so **100% of traffic goes to the legacy system**. The facade is in place but changes nothing. This is important: you deploy the facade first and verify it changes nothing, *before* you migrate anything. If something breaks at that point, you know it's the facade, not the new code.

Then you rebuild `createAccount` in the new system, test it, and update `newSystemReadyFor` to return `true` for `"createAccount"`. Now all account creation goes to the new system while deposits, withdrawals, and balance checks still go to legacy.

Repeat for each capability. Gradually the new system takes over.

The key structural insight: the only thing that changes at each cutover is what `newSystemReadyFor` returns. Clients don't change. The two systems don't change. In a real implementation that method reads a **feature flag** from configuration, so flipping a capability doesn't even require a deployment — and flipping it *back* takes seconds.

Part 8 — Key insights: the proxy layer is everything

The router/proxy is the heart of the pattern, and it needs to be genuinely robust — it's now on the critical path for **every single request**. The course lists three kinds of routing logic:

Feature flags. Turn new functionality on and off to test it. The operational superpower here is **instant rollback** (returning to the previous, known-good behaviour) — no deploy, no rebuild, just a config change.

User segmentation. Route certain users to new services for **A/B testing or phased rollouts** (A/B testing shows two versions to different user groups and compares results; a phased rollout releases to a growing share of users step by step). Common progressions: internal employees first, then 1% of users, then 10%, then everyone. Or by geography: a small market first, so problems affect fewer customers.

API versioning. Direct requests based on version **headers** (key-value metadata sent alongside an HTTP request, such as `Accept-Version: 2`) or **paths** (the part of the URL after the domain) — for example `/v1/orders` to legacy and `/v2/orders` to the new service.

Part 9 — Trade-offs and what actually goes wrong

Running two systems at once. You operate, monitor, and pay for both throughout the migration — which can last years. Two **deployment pipelines** (automated sequences that build, test and release code), two **on-call rotations** (schedules of engineers who respond to production incidents), two sets of **dashboards** (screens of live graphs showing system health).

The facade is a single point of failure (one component whose failure takes the whole system down). Everything flows through it. It needs **high availability** (staying up nearly all the time, usually via redundant copies) and a carefully considered **latency budget** (the maximum delay each component is allowed to add; **latency** is the time a request takes from start to finish — see System Design Fundamentals), because whatever time it adds is added to every request.

An extra hop. (A **hop** is one network trip between two machines.) The routing layer adds network latency to every request, however small.

Data consistency — the hard part the pattern doesn't solve. If old and new systems share or split data, you need a strategy: a shared database, synchronisation, or **dual writes** (the application writing the same change to two databases). This is genuinely the hardest part of most real migrations, and the pattern gives you no help with it.

> *Concrete example of why:* suppose you migrate "create account" to the new system but leave "check balance" on legacy. If the two write to different databases, an account created in the new system is invisible to the legacy balance check. In practice teams usually keep a **single shared database** during migration and split it later — a second strangler migration in its own right.

Duplicated logic. While a capability is in flight, the same business rule exists in two places. Change the rule and you must change both, or they diverge.

Migration fatigue — the most common real-world failure.

> The pattern makes it *safe* to stop halfway, so organisations do. The urgent 60% gets migrated, the pain goes away, priorities shift, and the team runs both systems forever — which is worse than either system alone. **Set a decommission date for the legacy system up front and treat it as a real deliverable.**

Part 10 — Common beginner misconceptions

"The new system must be microservices." (**Microservices** means building an application as many small, independently deployable services, each owning one business capability.) No. Strangler Fig is about *how* you migrate, not what you migrate *to*. You can strangle one monolith into a better monolith.

"You migrate the database first." Usually the opposite. Data migration is the riskiest part, so teams typically keep the shared database initially and move behaviour first, splitting data later.

"The facade is temporary." Often it isn't — it frequently becomes the permanent API Gateway. That's fine, as long as it's a deliberate choice.

"Once a capability is migrated you delete the old code immediately." Leave it in place, unreachable but intact, until you're confident. It's your rollback. Delete it once the flag has been at 100% for a sensible period.

"This is only for ancient systems." It works any time you replace a component that's in use — swapping a payment provider, replacing a search engine, moving from one cloud to another.

Part 11 — Check your understanding

1. Why is the strangler fig a better metaphor than "demolish and rebuild"?
2. What single method in the `BankingFacade` example controls the entire migration?
3. Why route by a *consistent* function of user ID rather than randomly per request?
4. Why deploy the facade before migrating anything?
5. What problem does the pattern explicitly *not* solve?
6. What's the most common organisational failure mode, and the countermeasure?

(Answers: 1 — both systems coexist and the transition is gradual and reversible; 2 — `newSystemReadyFor`; 3 — a user must get consistent behaviour, or differences look like intermittent bugs; 4 — to verify the facade itself is harmless before adding a second variable; 5 — data consistency between old and new; 6 — stalling halfway, prevented by committing to a decommission date.)

Part 12 — Interview soundbite (30 seconds)

> "Strangler Fig is how you replace a legacy system without a big-bang rewrite. You put a facade or proxy in front of the old system so clients only ever talk to that, then rebuild functionality one capability at a time behind it. The facade routes each request to either the new service or the legacy system depending on what's been migrated — usually driven by feature flags or user segmentation, so you get percentage rollouts and instant rollback without a deploy. Over time traffic to legacy drops to zero and you delete it. It's named after a tree that grows around its host until the host rots away. The reason it beats a rewrite is that a rewrite has to hit a moving target and discovers all the undocumented legacy behaviour on switchover day, whereas this surfaces problems one capability at a time while you still have a way back. The trade-offs are running both systems in parallel for a long time, the facade becoming a critical single point of failure, and data consistency between old and new, which the pattern doesn't solve. The biggest

> practical risk is organisational — because it's safe to stop halfway, teams often do, so you commit to a decommission date up front."

Part 13 — Implementation

A. Runnable demo

Full source: `demos/StranglerFigDemo.java`

```
cd demos && ./gradlew runCh02
```

(`./gradlew` is the **Gradle wrapper** — a script that downloads and runs **Gradle**, a build tool that compiles Java code and runs tasks such as this demo.)

The core is the flag check that makes rollout **consistent per user**:

```
boolean useNewSystem(String capability, String userId) {
    int percent = rolloutPercent.getOrDefault(capability, 0);
    if (percent <= 0)   return false;
    if (percent >= 100) return true;
    int bucket = Math.floorMod(userId.hashCode(), 100);   // stable per user
    return bucket < percent;
}
```

`Math.floorMod` (not `%`) because `hashCode()` (Java's built-in hash function for an object) can be negative and `%` would then yield a negative bucket that never matches.

B. What you should see

```
=== Step 1: facade deployed, nothing migrated ===
  LEGACY:created:alice
  >> all traffic still on LEGACY - facade proven harmless

=== Step 3: same user always gets the same system ===
  user-7 -> LEGACY:created:acct     (x3, never flips)

=== Step 4: widen the rollout ===
    [flags] createAccount -> 25%  ->  actual: 253/1000 users
    [flags] createAccount -> 50%  ->  actual: 493/1000 users
    [flags] createAccount -> 100% ->  actual: 1000/1000 users

=== Step 5: problem found - INSTANT rollback ===
  user-7 -> LEGACY:created:acct
  >> no deploy, no rebuild: config change only
```

Two things worth noting from the real output. **Step 3** proves consistency — user-7 gets the same system every time, so a behavioural difference can never look like an intermittent bug. And the percentages aren't exact (5% gave 44/1000, not 50) because **hash distribution is only approximately uniform** at these sample sizes. That's expected and fine; you're **canarying** (sending a small slice of real traffic to the new version first, like a canary in a coal mine that warns of danger early), not metering.

C. Production — routing at the proxy

In reality the facade is usually infrastructure, not application code.

Nginx — route by path: (**Nginx** is a popular web server and **reverse proxy** — a proxy that sits in front of servers and forwards client requests to them. An `upstream` block names a group of backend servers.)

```
upstream legacy_system { server legacy.internal:8080; }
upstream new_system    { server orders-v2.internal:8080; }

server {
    location /api/orders  { proxy_pass http://new_system;    }   # migrated
    location /api/billing { proxy_pass http://legacy_system; }   # not yet
    location /            { proxy_pass http://legacy_system; }   # default: legacy
}
```

Istio — percentage rollout without touching application code: (**Istio** is a **service mesh** — infrastructure that routes and secures traffic between services through proxies placed next to each one; it is configured with **YAML**, a plain-text, indentation-based config format. See Sidecar.)

```
apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata:
  name: orders
spec:
  hosts: [orders]
  http:
    # Internal staff always get the new system - your first canary group
    - match:
        - headers:
            x-user-group: { exact: internal }
      route:
        - destination: { host: orders, subset: v2 }
    # Everyone else: 5% to the new system
    - route:
        - destination: { host: orders, subset: v1 }
          weight: 95
        - destination: { host: orders, subset: v2 }
          weight: 5
```

Feature flags in application code — LaunchDarkly (a hosted feature-flag service), Unleash (an open-source equivalent), or a table in your **config store** (a central place where configuration values are kept):

```
if (flags.isEnabled("orders-v2", userContext)) {
    return newOrderService.create(order);
}
return legacyOrderService.create(order);
```

D. The part the pattern doesn't solve: data

The hardest real-world problem. Three strategies, in increasing order of difficulty:

1. Shared database (start here). Both systems read and write the same database. No sync problem at all — the new service simply uses the old schema. Split the data later as a separate migration.

2. Dual writes. The new system writes to both databases during transition. Simple to describe, genuinely hard to get right: if the second write fails you have divergence, and there's no **transaction** (a group of database changes that either all succeed or all fail together) spanning both.

3. Change Data Capture (CDC). (CDC means watching a database for every insert, update and delete and publishing each change as an event.) Stream changes from the legacy database into the new one using **Debezium** (an open-source CDC tool) reading the **transaction log** (the database's own append-only record of every change it commits). The new system's data is **eventually consistent** with legacy (it may lag briefly, but catches up if no new changes arrive), and crucially **the legacy system needs no modification**:

```
# Debezium connector: legacy Postgres -> Kafka -> new service
connector.class: io.debezium.connector.postgresql.PostgresConnector
database.hostname: legacy-db
table.include.list: public.orders,public.customers
plugin.name: pgoutput
```

E. Implementation pitfalls

1. Random rather than consistent routing. (The comment in the Debezium config above mentions **Postgres** — PostgreSQL, a popular open-source relational database — and **Kafka**, a distributed log that stores and streams events; see Event-Driven.) `Math.random() < 0.05` sends the *same user* to different systems on consecutive requests. Any behavioural difference becomes an unreproducible bug report. Always hash a stable identifier.

2. Deploying the facade and migrating a capability in one release. If something breaks you can't tell which change caused it. Deploy the facade first, confirm it's a no-op, then migrate.

3. Deleting legacy code the moment traffic hits zero. That code is your rollback. Leave it unreachable but intact until the flag has been at 100% for a full business cycle.

4. No comparison stage. Before flipping traffic, run **shadow mode** — send the request to both systems, return the legacy response, and log any differences. This is how you find undocumented behaviour *before* it reaches users. The code runs the new system **asynchronously** (in the background on another thread, so the user's request doesn't wait for it) and records **metrics** (numeric counters and measurements used for monitoring):

```
Result legacyResult = legacy.process(request);
executor.submit(() -> {                          // async, never blocks the user
    try {
        Result newResult = modern.process(request);
        if (!newResult.equals(legacyResult)) {
            metrics.increment("shadow.mismatch");
            log.warn("Divergence for {}: {} vs {}", request.id(), legacyResult, newResult);
        }
    } catch (Exception e) { metrics.increment("shadow.error"); }
});
return legacyResult;                             // user always gets the proven answer
```

5. Flags that outlive the migration. Every completed migration leaves a dead **conditional** (an `if` statement whose other branch can never be taken). Schedule flag removal as part of "done," or the codebase fills with permanent forks.

6. No decommission date. The organisational failure mode from Part 9. Put it in the plan.

Related patterns

- **API Gateway** — the facade is very often literally an API Gateway
- **BFF** — can bridge new frontends to legacy systems during migration
- **Config Externalization** — where the feature flags driving routing actually live